파이썬으로 머신러닝

스스로 진행하는 학습 가이드

비전공자를 위한 실전 안내서

Ver. 1.0

목차

1부. 머신러닝의 전체 흐름과 개념

1.1 시작하며 — 이 가이드가 지향하는 것
1.2 머신러닝이란 무엇인가
1.3 종속변수와 독립변수
1.4 머신러닝의 세 가지 카테고리
1.5 지도학습 상세: 분류와 회귀
1.6 비지도학습 상세: 군집·차원축소·이상탐지
1.7 강화학습 판단 기준
1.8 시계열 데이터의 위치
1.9 판단 프레임워크 종합
1.10 전체 워크플로 개요

2부. 전처리 상세

2.1 전처리가 왜 중요한가
2.2 첫 탐색 — 데이터 감 잡기
2.3 전처리의 실용 순서
2.4 분포 확인과 정규성 판단
2.5 종속변수와 독립변수의 관계 분석
2.6 독립변수끼리의 관계 — 다중공선성
2.7 컬럼 제거 판단 4가지 기준
2.8 특성 엔지니어링 — 원본에서 새 특성 뽑기
2.9 결측값 처리
2.10 범주형 인코딩
2.11 스케일링(정규화)
2.12 지도학습과 비지도학습의 전처리 차이
2.13 시계열 전처리 — 시간 특성 만들기

3부. 학습 상세

3.1 train/test 분리
3.2 지도학습 — 분류
3.3 지도학습 — 회귀
3.4 비지도학습 — 군집화
3.5 비지도학습 — 차원축소
3.6 비지도학습 — 이상탐지
3.7 시계열 접근
3.8 평가 지표 총정리
3.9 하이퍼파라미터 튜닝
3.10 파이프라인으로 정리하기
3.11 자주 겪는 함정

4부. 실전 워크플로와 AI 활용법

4.1 실전 워크플로 10단계
4.2 AI에게 물을 때 반드시 줘야 하는 정보
4.3 단계별 프롬프트 템플릿
4.4 AI 답변 검증 4단계

4.5 위험한 프롬프트 패턴
4.6 실전 세션 흐름 예시
4.7 요약 카드 — 한 장 정리
4.8 마무리 — 자기 프로젝트 시작하기

PART 1

머신러닝의 전체 흐름과 개념

코드에 손대기 전에 알아야 할 것들. 지도·비지도·강화학습이 무엇이고, 내 문제가 어디에 속하는지 판단하는 프레임을 익힙니다.

1부. 머신러닝의 전체 흐름과 개념

1.1 시작하며 — 이 가이드가 지향하는 것

머신러닝을 처음 배우는 사람 대부분이 알고리즘 이름에 압도됩니다. 로지스틱 회귀, 랜덤포레스트, XGBoost, K-Means, PCA, 그리고 딥러닝까지. 하지만 알고리즘을 하나하나 외운다고 실무를 할 수 있는 것은 아닙니다. 오히려 그 반대입니다.

중요한 것은 **흐름**입니다. 문제를 판단하고, 데이터를 다듬고, 적절한 모델을 골라 학습시키고, 결과를 평가하는 이 네 단계의 흐름만 몸에 배면 어떤 문제든 접근할 수 있습니다. 알고리즘은 그때그때 필요한 것을 찾아 쓰면 됩니다. 오늘날에는 생성형 AI가 그 조사를 도와주니까요.

이 가이드의 목표는 세 가지입니다.

1. 내 데이터를 앞에 두고 "지금 뭘 해야 하지?"라는 질문에 스스로 답할 수 있게 되기
2. 각 단계에서 무엇을 판단해야 하고, 그 판단의 근거가 무엇인지 알기
3. 막힐 때 AI에게 제대로 물어보고, 그 답을 검증할 수 있는 감각 기르기

이 가이드는 답을 주지 않습니다. 답에 도달하는 **방법**을 알려줍니다. 이 방식이 처음에는 답답할 수 있지만, 세 번째 프로젝트쯤 되면 왜 이렇게 배웠는지 이해하게 됩니다. 실무에서는 매번 데이터가 다르고, 매번 답이 다르기 때문입니다.

> **이 가이드를 활용하는 법.** 순서대로 한 번 읽고 나면, 그다음부터는 참고서처럼 씁니다. 자기 프로젝트를 시작할 때마다 목차에서 필요한 절만 다시 펴 보세요. 특히 4부의 프롬프트 템플릿은 프린트해서 옆에 두면 유용합니다.

1.2 머신러닝이란 무엇인가

머신러닝을 한 문장으로 정의하면 **"데이터에서 규칙을 자동으로 찾아내는 방법"**입니다.

전통적인 프로그래밍은 사람이 규칙을 코드로 적어 넣습니다. "매출이 100만 원을 넘으면 우수 고객으로 분류하라" 같은 규칙을 사람이 정해서 프로그램에 넣죠. 머신러닝은 반대입니다. 데이터와 원하는 결과를 주면 컴퓨터가 규칙을 스스로 찾아냅니다.

구분	전통 프로그래밍	머신러닝
입력	데이터 + 규칙	데이터 + 정답
출력	결과	규칙 (모델)
예시	매출액에 세율 곱하기	과거 매출로 다음 달 매출 예측

머신러닝이 잘 어울리는 문제는 다음과 같습니다.

- 규칙이 너무 복잡해 사람이 다 쓰기 어려운 경우 (스팸 메일 판별)
- 규칙이 계속 바뀌는 경우 (매출 예측 — 시장이 계속 변함)
- 데이터에서 사람이 발견하지 못한 패턴을 찾고 싶은 경우 (고객 세그멘테이션)

반대로 머신러닝을 쓸 필요가 없는 경우도 많습니다. 규칙이 명확한 문제(부가세 계산), 데이터가 너무 적은 경우(30건), 단순 규칙으로 충분한 경우는 그냥 코드를 쓰는 편이 낫습니다. "머신러닝이 필요한가"를 판단하는 것도 실력입니다.

1.3 종속변수와 독립변수

머신러닝을 하려면 먼저 **"내가 뭘 알아내려 하고, 무엇을 근거로 삼는가"**를 명확히 해야 합니다. 이 두 역할에 이름이 붙어 있습니다.

정의

종속변수 (Dependent Variable)는 예측하고 싶은 값입니다. 결과, 답에 해당합니다. 다른 변수들의 영향을 "받는" 쪽이라서 종속이라 부릅니다. 지도학습에서는 반드시 있어야 하는 컬럼이고, 코드에서는 보통 소문자 `y`로 표기합니다.

독립변수 (Independent Variable)는 예측에 사용하는 값들입니다. 원인, 근거에 해당합니다. 다른 변수에 "영향을 주는" 쪽이라 독립이라 부릅니다. 여러 개일 수 있으며, 코드에서는 대문자 `X`로 표기합니다. 대문자인 이유는 여러 컬럼이 모여 행렬(2차원)을 이루기 때문입니다.

예시

문제	종속변수 (y)	독립변수 (X)
집값 예측	집값	평수, 방 개수, 지하철 거리
이탈 예측	이탈 여부	가입 기간, 월 사용액, 상담 횟수
가스공급량 예측	가스공급량	기온, 요일, 공휴일 여부

같은 개념 다른 이름

머신러닝, 통계학, 실무에서 부르는 이름이 조금씩 달라 헷갈립니다. 다음 표의 각 행은 모두 같은 것을 가리킵니다.

종속변수의 다른 이름	독립변수의 다른 이름
Target (타깃)	Feature (특성, 피처)
y	X
Response variable (반응변수)	Predictor variable (설명변수)
Label (라벨)	Attribute (속성)
결과변수	원인변수

코드에서 나누는 방법

```
y = df['생존여부']                    # 종속변수
X = df.drop(columns=['생존여부'])     # 독립변수 (나머지 전부)

print(y.shape)   # (891,)       1차원
print(X.shape)   # (891, 10)    2차원 (행 × 컬럼)
```

> **비지도학습에는 종속변수가 없습니다.** 지도학습은 X → y의 관계를 배우는 반면, 비지도학습은 X만 가지고 구조를 찾습니다. 그래서 K-Means, PCA 같은 것을 할 때는 y가 존재하지 않습니다.

1.4 머신러닝의 세 가지 카테고리

머신러닝은 크게 세 갈래로 나뉩니다.

지도학습 (Supervised Learning)

정답이 있는 데이터로 정답을 맞히는 법을 배웁니다. 데이터 안에 "예측하고 싶은 값(종속변수)"이 이미 컬럼으로 존재합니다. 과거 6개월치 매출 데이터가 있고 다음 달 매출을 예측하고 싶다면, 과거 매출이 정답 역할을 합니다. 스팸 메일 예시가 있고 새 메일이 스팸인지 판별하고 싶다면, 과거 스팸/정상 라벨이 정답입니다.

비지도학습 (Unsupervised Learning)

정답 없는 데이터에서 숨은 구조를 찾습니다. 고객 명단이 있는데 비슷한 사람끼리 묶고 싶은 경우, 수십 개 변수를 두세 개로 압축해 시각화하고 싶은 경우, 평소와 다른 이상한 거래를 찾고 싶은 경우가 여기 해당합니다.

강화학습 (Reinforcement Learning)

환경과 상호작용하며 최적의 행동 전략을 배웁니다. 알파고, 자율주행, 로봇 제어가 대표적입니다. 실무에서 정형 데이터로 다루는 경우가 드물기 때문에, 이 가이드에서는 판단 기준만 다루고 실습은 하지 않습니다.

비교

구분	지도학습	비지도학습	강화학습
정답	있음	없음	보상(신호)만 있음
목적	예측	구조 발견	최적 전략 학습
대표 예	매출 예측, 이탈 판별	고객 세그멘테이션, 이상탐지	게임 AI, 로봇 제어
대표 알고리즘	RandomForest, XGBoost	K-Means, PCA	Q-Learning, DQN

1.5 지도학습 상세: 분류와 회귀

지도학습은 다시 두 갈래로 나뉩니다. 판단 기준은 **"예측하고 싶은 값이 숫자인가, 카테고리인가"**입니다.

회귀 (Regression)

연속된 숫자값을 예측합니다. 값이 얼마든 될 수 있고, 값 사이의 크기 비교가 의미 있습니다.

- 매출액 예측: 100만 원, 200만 원, 234만 5천 원 등 어떤 값도 가능
- 집값 예측, 대기시간 예측, 온도 예측
- 가스공급량 예측 (시계열 회귀)

분류 (Classification)

정해진 몇 개의 카테고리 중 하나를 예측합니다. 값 사이에 크기 비교가 반드시 의미 있는 것은 아닙니다.

- 이진분류: 스팸/정상, 이탈/유지, 양성/음성 (2개 클래스)
- 다중분류: A등급/B등급/C등급, 강아지/고양이/새 (3개 이상)

회귀냐 분류냐, 판단하는 법

예측 대상의 컬럼 값을 직접 보세요.

```
print(y.dtype)          # int64, float64 → 회귀 가능성
                        # object, category → 분류
print(y.nunique())      # 유니크 값 개수
print(y.value_counts())
```

유니크 값이 2~10개 정도의 정해진 범주면 분류, 연속적으로 다양한 숫자값이면 회귀입니다. 애매한 경우는 "이 값들 사이의 크기 차이가 의미 있는

가?"를 스스로에게 물어보세요. "5등급이 3등급의 5/3배 만족스러운가?"는 이상한 질문입니다. 이런 경우는 분류로 취급하는 편이 낫습니다.

1.6 비지도학습 상세: 군집·차원축소·이상탐지

비지도학습은 목적을 먼저 뾰족하게 정해야 합니다. 다음 세 가지 중 무엇을 하고 싶은지가 판단의 시작입니다.

군집화 (Clustering)

비슷한 것끼리 묶습니다.

- 고객 세그멘테이션: 소비 패턴이 비슷한 고객을 그룹으로 묶어 마케팅에 활용
- 지역 유형 분석: 인구·소득·상권 지표가 비슷한 동네를 묶기
- 제품 카테고리 발견

대표 알고리즘: **K-Means**, DBSCAN, 계층적 군집화.

차원축소 (Dimensionality Reduction)

변수의 개수를 줄입니다. 시각화, 노이즈 제거, 학습 속도 향상이 목적입니다.

- 수십 개의 설문 문항을 두세 개의 성향 축으로 요약
- 50차원 데이터를 2차원으로 압축해 산점도로 시각화

대표 알고리즘: **PCA** (선형), t-SNE·UMAP (시각화 전용).

이상탐지 (Anomaly Detection)

평소와 다른 이상한 데이터를 찾습니다.

- 신용카드 이상 거래 탐지
- 공장 설비의 이상 신호 감지
- 서버 접속 로그의 침입 시도 탐지

대표 알고리즘: **IsolationForest**, OneClassSVM, LOF.

1.7 강화학습 판단 기준

강화학습을 써야 하는 경우는 실제로 드뭅니다. 정형 데이터를 다루는 대부분의 업무는 지도학습이나 비지도학습으로 풀립니다. 강화학습을 고려할 때는 다음 세 조건을 **모두 만족**해야 합니다.

1. **순차적 결정.** 지금 한 행동이 다음 상태에 영향을 줍니다. 한 판이 끝나고 다시 결정하는 게 아니라, 연쇄적으로 이어져야 합니다.
2. **보상 신호가 명확.** 좋고 나쁨을 숫자로 정의할 수 있어야 합니다. "게임에서 이기면 +1, 지면 -1" 같은.
3. **시뮬레이션이 가능하거나 실패 비용이 낮음.** 강화학습은 수많은 시행착오로 배우기 때문입니다. 실패할 때마다 큰 비용이 들면 못 씁니다.

강화학습으로 착각하기 쉬운 문제들

문제	얼핏 보면	실제로는
고객 이탈 예측	강화학습?	이진분류 (지도학습)
다음 달 발주량 결정	강화학습?	회귀로 먼저 시도 (지도학습)
주식 매수 시점	강화학습?	과거 데이터 있으면 지도학습부터
추천 시스템	강화학습?	협업 필터링(비지도) 또는 분류

진짜 강화학습이 어울리는 예: 게임 AI, 로봇 제어, 자율주행, 실시간 광고 입찰, 창고 로봇의 물류 최적화.

1.8 시계열 데이터의 위치

많은 사람이 "시계열"을 별도 카테고리로 오해합니다. 시계열은 알고리즘의 종류가 아니라 **데이터의 형태**입니다. 시간 순서가 중요한 데이터라는 뜻이죠.

시계열 데이터로 값을 예측하면 회귀가 되고, 카테고리를 예측하면 분류가 됩니다. 즉 시계열은 지도학습 안에 들어가 있는 형태입니다.

```
지도학습
├─ 회귀 (숫자 예측)
│   ├─ 일반 회귀 (집값, 매출액)   ← 시간 무관
│   └─ 시계열 회귀 (가스공급량)   ← 시간 중요
└─ 분류 (카테고리 예측)
    ├─ 일반 분류 (이탈/유지)
    └─ 시계열 분류 (다음 시간 이상 여부)
```

다만 시계열은 다루는 방식이 특별합니다. 세 가지가 다릅니다. train/test 분리를 시간 순서대로 해야 하고, 시간에서 특성을 뽑아내는 작업이 필수이며, ARIMA·Prophet 같은 전용 모델이 별도로 있습니다. 자세한 것은 2부와 3부에서 다룹니다.

1.9 판단 프레임워크 종합

여기까지 배운 것을 하나의 판단 흐름으로 정리하면 다음과 같습니다. 자기 데이터를 가지고 이 순서대로 답하면 됩니다.

학생용 판단 시트

1. 내 데이터에 "예측하고 싶은 정답 컬럼"이 있는가?
   - 있음 → 지도학습
   - 없음 → 비지도학습 또는 강화학습 후보
2. 지도학습이라면: 그 컬럼이 숫자인가 카테고리인가?
   - 숫자 → 회귀
   - 카테고리 → 분류
3. 비지도학습이라면: 나는 (a) 묶기 (b) 줄이기 (c) 이상찾기 중 무엇을 하려는가?
4. 이 데이터에 시간 순서가 중요한가?
   - 중요함 → 시계열 접근 (train/test 시간 순 분리, 시간 특성 만들기)
   - 중요하지 않음 → 일반 접근
5. 강화학습 3조건(순차 결정, 명확한 보상, 시뮬레이션 가능)을 모두 만족하는가?
   - 모두 만족 → 강화학습 고려
   - 아니면 → 위 판단으로 돌아가기

1.10 전체 워크플로 개요

문제 판단이 끝났으면 다음 순서로 진행합니다. 이 흐름은 어떤 문제든 큰 틀에서 동일합니다.

1. **데이터 로드와 첫 탐색** — shape, head, info, describe
2. **종속변수와 독립변수 분리**
3. **1차 컬럼 정리 + 특성 엔지니어링** — 명백히 지울 것 정리, 원본 못 쓰는 컬럼에서 새 특성 뽑기
4. **분포 확인** — 남은 수치형 컬럼에 대해
5. **관계 분석** — X와 y, X들끼리
6. **2차 컬럼 정리** — 관계 없는 것 추가 제거
7. **결측값 처리**
8. **인코딩** — 범주형을 숫자로
9. **스케일링** — 필요한 모델만
10. **train/test 분리**
11. **모델 학습** — baseline부터 시작

2. **평가** — 문제 성격에 맞는 지표로
3. **개선** — 하이퍼파라미터 튜닝, 특성 추가

2부에서는 1~9번(전처리), 3부에서는 10~13번(학습)을 자세히 다룹니다. 4부에서는 이 전체를 실전 시나리오와 AI 활용법으로 엮어 봅니다.

> **순서에 대한 팁.** "지울 컬럼의 분포를 미리 확인하는 건 시간 낭비"입니다. 그래서 명백한 제거를 먼저 하고, 남은 것에 대해서만 분포와 관계를 봅니다. 그리고 관계 분석 결과가 두 번째 제거 판단의 근거가 됩니다. 이 실용 순서는 2.3절에서 다시 자세히 다룹니다.

P A R T 2

전처리 상세

"좋은 데이터에 평범한 모델 > 나쁜 데이터에 좋은 모델"입니다. 전처리에서 결과의 80퍼센트가 결정됩니다.

2부. 전처리 상세

2.1 전처리가 왜 중요한가

실무에서 머신러닝 프로젝트의 시간 배분을 물어보면 "80퍼센트가 데이터 정리, 20퍼센트가 모델링"이라는 답이 흔합니다. 처음 배울 때는 이 말이 잘 안 와닿습니다. 화려한 알고리즘이 다 해줄 것 같으니까요. 하지만 몇 번 프로젝트를 해보면 이 말이 진리라는 걸 알게 됩니다.

이유는 간단합니다. 모델은 여러분이 준 데이터 안에서만 배웁니다. 데이터가 잘못돼 있으면 아무리 좋은 모델을 써도 결과는 잘못 나옵니다. 반대로 데이터가 잘 정리돼 있으면 평범한 모델(로지스틱 회귀, 랜덤포레스트)만으로도 충분히 좋은 성능이 나옵니다.

전처리는 정형화된 정답이 없는 부분입니다. 데이터에 따라 판단이 달라지고, 같은 데이터라도 사람에 따라 방식이 다릅니다. 하지만 **판단의 기준**은 정해져 있습니다. 이 2부는 그 기준들을 정리한 것입니다.

2.2 첫 탐색 — 데이터 감 잡기

코드부터 짜지 마세요. 먼저 데이터를 눈으로 봐야 합니다.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 한글 폰트 세팅 (윈도우 기준)
plt.rcParams['font.family'] = 'Malgun Gothic'
plt.rcParams['axes.unicode_minus'] = False

df = pd.read_csv('내_데이터.csv')

df.shape                # (행 수, 컬럼 수)
df.head()               # 앞 5행 - 어떤 데이터인지 감 잡기
df.info()               # 컬럼별 타입, 결측치 요약
df.describe()           # 수치형 통계 요약
df.isnull().sum()       # 컬럼별 결측치 개수
df.dtypes               # 컬럼별 데이터 타입
```

이 여섯 줄만 실행해도 데이터의 인상이 대략 잡힙니다. 특히 봐야 할 것들은 다음과 같습니다.

- **행과 열의 규모.** 100행 미만이면 머신러닝보다 통계·규칙 기반이 나을 수 있습니다.
- **컬럼별 타입.** 수치형과 범주형이 몇 개씩인지.
- **결측치 분포.** 어느 컬럼이 얼마나 비어 있는지.
- **수치형의 범위와 스케일.** 어떤 컬럼은 0~1, 어떤 컬럼은 0~10000이라면 스케일링이 필요합니다.
- **이상해 보이는 값.** 나이가 300살, 매출액이 음수 같은 것.

2.3 전처리의 실용 순서

전처리 단계를 무작정 순서대로 다 하면 시간이 낭비됩니다. 예를 들어 결국 지워버릴 컬럼의 분포를 확인하는 건 헛수고입니다. 실무에서는 다음 순서로 진행하는 편이 효율적입니다.

> **효율적인 전처리 순서**
>
> 1. **첫 탐색** — shape, head, info, describe (2.2)
> 2. **1차 컬럼 정리** — 명백히 지울 것 먼저 정리
>   · 식별자 (회원번호 등)
>   · 결측 50퍼센트 이상
>   · 값이 거의 하나인 컬럼
>   · 원본은 못 쓰지만 **가공하면 유용한 컬럼은 이 단계에서 뽑아냄** (2.8 참고)
> 3. **남은 수치형 컬럼의 분포 확인** (2.4)
> 4. **관계 분석** — X와 y, X끼리 (2.5, 2.6)
> 5. **2차 컬럼 정리** — 관계가 약한 것, 다중공선성 대응 (2.7)
> 6. **결측값 처리** (2.9)
> 7. **인코딩** (2.10)
> 8. **스케일링** (2.11)

왜 이 순서인가

지울 컬럼의 분포를 미리 확인하면 시간 낭비입니다. 20개 컬럼 중 5개가 명백한 식별자·결측 다수 컬럼이라면, 이 5개는 분포를 볼 이유가 없습니다. 먼저 정리하고 나머지 15개에 대해서만 분포·관계를 봐야 합니다.

단, 한 가지 예외가 있습니다. **원본은 못 쓰지만 가공하면 유용한 컬럼**이 있습니다. 예를 들어 타이타닉 데이터의 Name 컬럼은 유니크 값이 891개라 언뜻 식별자처럼 보이지만, 호칭(Mr./Mrs./Miss)만 뽑으면 유용한 새 특성이 됩니다. 이런 컬럼을 미리 지워버리면 정보를 잃습니다. 그래서 1차 컬럼 정리 단계에서 "이 컬럼을 지우기 전에 뽑아낼 정보가 있는지"를 반드시 검토해야 합니다. 자세한 내용은 2.8절에서 다룹니다.

왜 관계 분석이 컬럼 판단보다 앞에 오나

관계 분석 결과가 컬럼 제거의 근거가 되기 때문입니다. "이 컬럼이 타깃과 관계있는가"를 알아야 지울지 남길지 판단할 수 있습니다. 그래서 관계 분석 → 나머지 컬럼 정리 순서가 자연스럽습니다.

2.4 분포 확인과 정규성 판단

수치형 컬럼을 다룰 때 가장 먼저 봐야 하는 것이 **분포**입니다. 분포가 어떻게 생겼느냐에 따라 결측값 대체 방법, 스케일링 방법, 심지어 모델 선택까지 달라집니다.

어떤 컬럼에 대해 확인하는가

정규성 확인의 대상은 **수치형 컬럼 전부**입니다. 독립변수(X)든 종속변수(y)든 상관없습니다. 단, 예외가 하나 있습니다. **분류 문제의 종속변수는 카테고리 값이라 정규성 개념이 적용되지 않습니다.** 이 경우 대신 클래스 균형(각 값의 비율)을 봅니다.

확인의 목적은 대상에 따라 조금씩 다릅니다.

대상	언제	왜 보나
독립변수(X) 수치형	항상	스케일링 방법, 결측 대체 방법, 이상치 판단
종속변수(y) — 회귀	회귀 문제일 때	심하게 치우쳤으면 로그 변환 후 예측
종속변수(y) — 분류	안 봄	대신 클래스 균형 확인
잔차(residual)	선형 회귀 학습 후	모델 진단 (3부 참고)

> **한 문장 원칙.** 수치형 컬럼은 모두 분포를 확인한다. 독립변수든 종속변수든. 단, 분류의 종속변수는 카테고리라 정규성이 아니라 클래스 균형을 확인한다.

이 절에서는 주로 독립변수의 분포를 다룹니다. 회귀에서 종속변수의 로그 변환 사례는 3.3절, 잔차 진단은 3.11절에서 다시 다룹니다.

왜 분포를 봐야 하는가

판단	정규분포에 가까울 때	치우친 분포일 때
결측값 대체	평균	중앙값
스케일링	StandardScaler	RobustScaler 또는 로그 변환
이상치 판정	평균±3σ 기준	IQR 기준
선형 모델 가정	안전	변환 후 사용 권장

시각화로 분포 확인

가장 기본은 히스토그램입니다. 값의 빈도를 막대로 표현합니다.

```
# 히스토그램 하나
sns.histplot(df['Age'], kde=True, bins=30)
plt.title('나이 분포')
plt.show()

# 모든 수치형 컬럼을 한 번에
df.hist(figsize=(15, 10), bins=30)
plt.tight_layout()
plt.show()
```

히스토그램에서 봐야 할 것들:

- **좌우 대칭인가** — 대칭이면 정규분포에 가까움
- **어느 쪽으로 치우쳐 있는가** — 오른쪽 꼬리(길게 뻗음)가 흔함 (매출, 소득, 요금 같은 것)
- **봉우리가 몇 개인가** — 봉우리 두 개(이봉분포)는 데이터에 두 그룹이 섞여 있다는 신호
- **극단값이 있는가** — 오른쪽 끝에 홀로 있는 점들

Q-Q Plot — 정규성 판단의 표준

Q-Q 플롯(Quantile-Quantile Plot)은 데이터의 분위수와 정규분포의 분위수를 비교하는 그림입니다. 점들이 직선 위에 놓이면 정규분포에 가깝고, 벗어나면 정규분포가 아닙니다.

```
from scipy import stats

fig, ax = plt.subplots(figsize=(6, 6))
stats.probplot(df['Age'].dropna(), dist='norm', plot=ax)
ax.set_title('Q-Q Plot: Age')
plt.show()
```

해석 방법:

- 점이 직선을 잘 따라감 → 정규분포에 가까움
- 양 끝이 위로 휘어짐 → 오른쪽 꼬리가 긴 분포 (양의 왜도)
- 양 끝이 아래로 휘어짐 → 왼쪽 꼬리가 긴 분포 (음의 왜도)
- S자 곡선 → 꼬리가 정규분포보다 두꺼움

수치로 확인 — 왜도와 첨도

눈으로 볼 뿐 아니라 숫자로도 확인할 수 있습니다.

```
print(f"왜도(Skewness): {df['Age'].skew():.3f}")
print(f"첨도(Kurtosis): {df['Age'].kurt():.3f}")
```

왜도(Skewness)는 분포의 좌우 대칭성을 나타냅니다.

- 0에 가까움: 좌우 대칭 (정규분포에 가까움)
- 양수: 오른쪽 꼬리가 김 (매출, 소득처럼 큰 값 소수)
- 음수: 왼쪽 꼬리가 김
- 절댓값 1 이상: 심한 왜도. 변환을 고려

첨도(Kurtosis)는 분포의 뾰족함을 나타냅니다.

- 0에 가까움: 정규분포와 비슷
- 양수: 정규분포보다 뾰족함, 이상치 많음
- 음수: 정규분포보다 완만함

통계 검정 — Shapiro-Wilk 검정

정규성 여부를 통계적으로 검증하고 싶다면 Shapiro-Wilk 검정을 씁니다. p-value가 0.05보다 크면 정규분포로 봐도 무방합니다.

```
from scipy.stats import shapiro

stat, p = shapiro(df['Age'].dropna())
print(f"통계량: {stat:.4f}, p-value: {p:.4f}")

if p > 0.05:
    print("정규분포로 봐도 됩니다")
else:
    print("정규분포가 아닙니다")
```

주의. Shapiro-Wilk 검정은 데이터가 많으면(5000행 이상) 아주 작은 차이도 "정규분포 아님"으로 판단합니다. 데이터가 많을 때는 검정보다 히스토그램·Q-Q 플롯으로 시각적으로 판단하는 편이 실용적입니다.

정규분포가 아닐 때 대응

정규분포가 아니라고 반드시 문제인 것은 아닙니다. 다음 세 가지 선택지가 있습니다.

선택 1: 그대로 두고 트리 모델을 씀. 랜덤포레스트, XGBoost 같은 트리 계열은 분포에 민감하지 않습니다. 정규성 걱정 없이 그대로 쓸 수 있습니다.

선택 2: 로그 변환. 오른쪽으로 치우친 분포에 가장 자주 쓰는 방법입니다.

```
# log(1+x)를 씀 - x가 0이어도 안전
df['Fare_log'] = np.log1p(df['Fare'])

# 변환 전후 비교
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.histplot(df['Fare'], ax=axes[0], bins=30)
axes[0].set_title('변환 전')
sns.histplot(df['Fare_log'], ax=axes[1], bins=30)
axes[1].set_title('로그 변환 후')
plt.show()
```

선택 3: Box-Cox 또는 Yeo-Johnson 변환. 로그보다 더 유연한 정규화 변환입니다.

```
from sklearn.preprocessing import PowerTransformer

pt = PowerTransformer(method='yeo-johnson')
df['Fare_normal'] = pt.fit_transform(df[['Fare']])
# Yeo-Johnson은 음수도 처리 가능. Box-Cox는 양수만.
```

이상치 판정

분포를 볼 때 자연스럽게 이상치가 눈에 띕니다. Boxplot이 유용합니다.

```
sns.boxplot(x=df['Fare'])
plt.show()

# IQR 기준 이상치 찾기
Q1 = df['Fare'].quantile(0.25)
Q3 = df['Fare'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers = df[(df['Fare'] < lower) | (df['Fare'] > upper)]
print(f"이상치 개수: {len(outliers)}")
```

이상치를 발견했다면 다음을 고민합니다.

- **진짜 이상한 값인가?** 나이 300살, 매출 음수 → 데이터 오류. 제거 또는 수정.
- **드물지만 진짜 값인가?** 초고소득 고객 → 제거하면 정보 손실. 로그 변환이나 RobustScaler 사용.
- **이상 자체가 예측 대상인가?** → 이상탐지 문제로 재정의.

2.5 종속변수와 독립변수의 관계 분석

어떤 독립변수가 종속변수를 잘 설명할까요? 이걸 알아야 어떤 컬럼을 살릴지 판단할 수 있습니다. 관계 분석 방법은 **두 변수의 타입 조합**에 따라 달라집니다.

독립변수 타입	종속변수 타입	대표 방법
수치형	수치형	산점도, 상관계수
수치형	범주형	Boxplot, t-test, ANOVA
범주형	수치형	Boxplot, groupby 평균
범주형	범주형	교차표, 카이제곱

케이스 1: 수치형 × 수치형

예: 광고비와 매출액, 기온과 가스공급량.

첫 번째로 그릴 것 — 산점도. 산점도로만 알 수 있는 것들이 있습니다: 관계의 모양(직선인가 곡선인가), 이상치, 군집, 관계 없음(구름처럼 흩어짐).

```
sns.scatterplot(x='광고비', y='매출액', data=df)
plt.show()

# 추세선까지 추가하면 관계 판단이 쉬움
sns.regplot(x='광고비', y='매출액', data=df,
            line_kws={'color': 'red'})
plt.show()
```

수치로 확인 — 상관계수. Pearson은 선형 관계, Spearman은 순위 기반(비선형도 포착).

```
from scipy.stats import pearsonr, spearmanr

r, p = pearsonr(df['광고비'], df['매출액'])
print(f"Pearson 상관계수: {r:.3f}, p-value: {p:.4f}")

r_s, p_s = spearmanr(df['광고비'], df['매출액'])
print(f"Spearman 상관계수: {r_s:.3f}, p-value: {p_s:.4f}")
```

상관계수 해석 기준(절댓값):

- 0.7 이상: 매우 강한 관계
- 0.4 ~ 0.7: 중간 정도 관계

- 0.2 ~ 0.4: 약한 관계
- 0.2 미만: 거의 관계 없음

케이스 2: 수치형 × 범주형

예: 나이와 생존여부, 월 사용액과 이탈여부.

첫 번째로 그릴 것 — Boxplot. 그룹별 분포 위치를 비교합니다.

```
sns.boxplot(x='생존여부', y='나이', data=df)
plt.show()

# 분포 모양까지 보고 싶으면 Violin plot
sns.violinplot(x='생존여부', y='나이', data=df)
plt.show()
```

두 그룹의 상자가 확 다른 위치에 있으면 관계가 있는 것이고, 겹치면 관계가 약합니다.

수치로 확인 — 그룹별 통계.

```
df.groupby('생존여부')['나이'].agg(['mean', 'median', 'std'])
```

통계 검정 — t-test 또는 ANOVA.

```
from scipy.stats import ttest_ind, f_oneway

# 2개 그룹 비교 (t-test)
group1 = df[df['생존여부']==1]['나이'].dropna()
group2 = df[df['생존여부']==0]['나이'].dropna()
t, p = ttest_ind(group1, group2)
print(f"p-value: {p:.4f}")
# p < 0.05면 두 그룹 나이 차이가 통계적으로 유의미

# 3개 이상 그룹 비교 (ANOVA)
groups = [df[df['등급']==g]['나이'].dropna() for g in df['등급'].unique()]
f, p = f_oneway(*groups)
print(f"p-value: {p:.4f}")
```

케이스 3: 범주형 × 범주형

예: 성별과 생존여부, 지역과 이탈여부.

첫 번째로 볼 것 — 교차표.

```
# 절대 수치
pd.crosstab(df['성별'], df['생존여부'])

# 비율 (성별 안에서 생존 비율)
pd.crosstab(df['성별'], df['생존여부'], normalize='index')
# → 여성 생존율 74%, 남성 19% 같은 게 바로 보임
```

시각화 — Grouped bar chart, Heatmap.

```
# 그룹 나란히
sns.countplot(x='성별', hue='생존여부', data=df)

# 범주가 많을 때는 heatmap
ct = pd.crosstab(df['지역'], df['등급'])
sns.heatmap(ct, annot=True, fmt='d', cmap='Blues')
```

통계 검정 — 카이제곱.

```
from scipy.stats import chi2_contingency

ct = pd.crosstab(df['성별'], df['생존여부'])
chi2, p, dof, expected = chi2_contingency(ct)
print(f"p-value: {p:.4f}")   # p < 0.05면 관계 있음
```

관계 강도까지 알고 싶다면 Cramér's V를 씁니다(0~1, 클수록 강함).

```
def cramers_v(ct):
    chi2 = chi2_contingency(ct)[0]
    n = ct.values.sum()
    r, k = ct.shape
    return np.sqrt(chi2 / (n * (min(r, k) - 1)))

print(cramers_v(ct))
```

시각화와 통계 검정의 관계

> **원칙: 먼저 그림으로 보고, 그다음 숫자로 확인한다.**

숫자만 믿으면 안 되는 유명한 사례가 **앤스컴 콰르텟(Anscombe's Quartet)**입니다. 네 개의 데이터셋이 평균, 분산, 상관계수까지 다 똑같지만 그림으로 그리면 완전히 다른 모양입니다. 하나는 직선, 하나는 곡선, 하나는 이상치 하나가 상관계수를 만들어낸 것.

```
df_ans = sns.load_dataset("anscombe")
sns.lmplot(x="x", y="y", col="dataset", data=df_ans,
           col_wrap=2, height=4)
plt.show()
```

반대로 그림만 보고 관계 있다고 단정해도 안 됩니다. 30행짜리 데이터에서 산점도가 우상향으로 보여도 우연일 수 있습니다. p-value로 확인해야 합니다.

정리하면 두 가지를 다 해야 합니다.

- 시각화 → 관계의 모양, 이상치, 예상 못한 패턴 발견
- 통계 검정 → 관계가 우연이 아닌지 확증, 강도 수치화

2.6 독립변수끼리의 관계 — 다중공선성

관계 분석의 두 번째 관점입니다. X와 y의 관계뿐 아니라 **X들끼리의 관계**도 봐야 합니다. 특성들이 서로 너무 겹치면 문제가 생깁니다.

다중공선성이란

두 특성이 상관계수 0.9라면 사실상 같은 정보입니다. 예를 들어 "키(cm)"와 "키(inch)"가 함께 있으면 완전 중복입니다. 실무에서는 이렇게 명백하지 않지만, "월 소득"과 "연 소득", "평수"와 "방 개수" 같은 것들이 강한 상관을 보일 수 있습니다.

영향은 모델에 따라 다릅니다.

- **선형 모델 (LogisticRegression, LinearRegression)**: 계수가 불안정해집니다. "이 변수의 영향력이 얼마"라는 해석이 어려워집니다.
- **트리 모델 (RandomForest, XGBoost)**: 성능에는 큰 영향이 없지만, 특성 중요도가 두 변수에 나눠져 해석이 어려워집니다.

상관관계 히트맵으로 진단

```
plt.figure(figsize=(10, 8))
sns.heatmap(df.corr(numeric_only=True), annot=True,
            cmap='coolwarm', center=0, fmt='.2f')
plt.show()

# 0.9 넘는 쌍 찾기
corr_matrix = df.corr(numeric_only=True).abs()
upper = corr_matrix.where(
    np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
high_corr = [(col, row, upper.loc[row, col])
             for col in upper.columns for row in upper.index
             if pd.notnull(upper.loc[row, col])
             and upper.loc[row, col] > 0.9]
print(high_corr)
```

VIF(분산팽창인수)로 더 정확히

상관계수는 두 변수만 비교하지만, VIF는 "한 변수가 나머지 전체와 얼마나 겹치냐"를 봅니다.

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

vif = pd.DataFrame()
vif['특성'] = X.columns
vif['VIF'] = [variance_inflation_factor(X.values, i)
              for i in range(X.shape[1])]
print(vif.sort_values('VIF', ascending=False))
```

VIF 해석 기준:

- VIF < 5: OK
- 5 ~ 10: 주의
- 10 이상: 다중공선성 심각. 제거 고려

대응 방법

1. **겹치는 두 변수 중 하나를 제거.** 도메인 지식으로 어느 쪽이 더 의미 있는지 판단.
2. **두 변수를 하나로 합치기.** 평균이나 합계로.
3. **PCA로 차원 축소.** 여러 변수를 하나의 주성분으로 압축.
4. **그대로 두고 트리 모델 사용.** 성능만 신경 쓴다면.

2.7 컬럼 제거 판단 4가지 기준

어떤 컬럼을 지울지는 네 가지 기준으로 판단합니다.

기준 1: 식별자 성격

회원번호, 주문번호처럼 각 행마다 다른 값이라 예측에 쓸 정보가 없는 경우.

```
for col in df.columns:
    unique_ratio = df[col].nunique() / len(df)
    if unique_ratio > 0.95:
        print(f"{col}: 유니크 비율 {unique_ratio:.2%} → 제거 후보")
```

기준 2: 결측치가 너무 많음

50퍼센트 넘으면 제거하거나, "결측 여부" 자체를 새 컬럼으로 만듭니다. 결측 자체가 정보일 수 있기 때문입니다.

```
missing = df.isnull().sum() / len(df) * 100
print(missing.sort_values(ascending=False))

# 예: 캐빈 정보 유무 자체를 특성으로
df['HasCabin'] = df['Cabin'].notnull().astype(int)
df = df.drop(columns=['Cabin'])
```

기준 3: 값이 거의 하나

최빈값이 95퍼센트 넘으면 정보량이 거의 없습니다.

```
for col in df.columns:
    top_ratio = df[col].value_counts(normalize=True).iloc[0]
    if top_ratio > 0.95:
        print(f"{col}: 최빈값 비율 {top_ratio:.2%} → 제거 후보")
```

기준 4: 타깃과 관계없음

2.4에서 다룬 관계 분석에서 타깃과 관계가 극히 낮은 컬럼입니다. 가장 실용적인 방법은 랜덤포레스트에 넣고 특성 중요도를 뽑는 것입니다.

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(random_state=42).fit(X, y)
importance = pd.Series(rf.feature_importances_, index=X.columns)
importance.sort_values().plot(kind='barh', figsize=(8, 6))
plt.show()
# 중요도 0.01 미만이면 제거 후보
```

> **제거는 마지막 선택입니다.** "숫자로 못 바꿔서 지운다"가 아니라 "정보가 없어서 지운다"여야 합니다. 원본은 못 쓰지만 가공하면 유용한 컬럼이 많습니다. 다음 절(2.8)에서 특성 엔지니어링을 다룹니다.

2.8 특성 엔지니어링 — 원본에서 새 특성 뽑기

특성 엔지니어링은 **기존 컬럼에서 새 컬럼을 만드는 작업**입니다. 실무에서 모델 성능을 크게 올리는 가장 확실한 방법입니다. 화려한 알고리즘보다 좋은 특성 하나가 더 강력한 경우가 많습니다.

왜 필요한가

많은 원본 컬럼이 "그대로는 못 쓰지만 뜯어보면 유용"합니다. 몇 가지 대표 사례를 보겠습니다.

사례 1: 이름에서 호칭 뽑기

타이타닉의 Name 컬럼은 유니크 값이 891개(전체 행 수와 같음)라 언뜻 식별자처럼 보입니다. 그래서 그냥 지우기 쉽지만, 호칭(Title) 부분만 뽑으면 강력한 특성이 됩니다. 호칭은 나이·성별·사회적 위치 정보를 압축해 담고 있기 때문입니다.

```
# "Braund, Mr. Owen Harris" → "Mr"
df['Title'] = df['Name'].str.extract(' ([A-Za-z]+)\.', expand=False)

# 드문 호칭을 하나로 통합
df['Title'] = df['Title'].replace(
    ['Lady', 'Countess', 'Sir', 'Jonkheer'], 'Rare')
df['Title'] = df['Title'].replace(['Mlle', 'Ms'], 'Miss')
df['Title'] = df['Title'].replace('Mme', 'Mrs')

print(df['Title'].value_counts())
# Mr      517, Miss  185, Mrs   126, Master 40, Rare 23
```

사례 2: 날짜 분해

날짜 컬럼 하나는 원본으로는 예측에 못 씁니다. 하지만 여러 특성으로 분해할 수 있습니다.

```
df['date'] = pd.to_datetime(df['date'])

df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['day'] = df['date'].dt.day
df['dayofweek'] = df['date'].dt.dayofweek
df['is_weekend'] = (df['dayofweek'] >= 5).astype(int)
df['quarter'] = df['date'].dt.quarter
df['week'] = df['date'].dt.isocalendar().week

# 도메인 지식이 있으면 더 특별한 특성
df['is_month_end'] = df['date'].dt.is_month_end.astype(int)
df['days_from_holiday'] = ...   # 공휴일까지 남은 일수
```

사례 3: 주소·URL·이메일 파싱

```
# 주소에서 시/도만 추출
df['city'] = df['address'].str.split(' ').str[0]

# 이메일에서 도메인 추출
df['email_domain'] = df['email'].str.split('@').str[1]

# URL에서 도메인 추출
from urllib.parse import urlparse
df['url_domain'] = df['url'].apply(lambda x: urlparse(x).netloc)
```

사례 4: 수치 컬럼 조합

기존 수치형 컬럼끼리 조합해 새 컬럼을 만듭니다. 도메인 지식이 있으면 강력합니다.

```
# 타이타닉 - 가족 크기
df['FamilySize'] = df['SibSp'] + df['Parch'] + 1
df['IsAlone'] = (df['FamilySize'] == 1).astype(int)

# 매출 데이터 - 단가
df['UnitPrice'] = df['Revenue'] / df['Quantity']

# 부동산 - 방당 평수
df['AreaPerRoom'] = df['Area'] / df['Rooms']

# 시간 데이터 - 두 시점 차이
df['DaysBetween'] = (df['end_date'] - df['start_date']).dt.days
```

사례 5: 구간화 (Binning)

연속값을 구간으로 나눠 범주형처럼 다루는 방법입니다. 나이나 소득처럼 구간별 의미가 다른 경우 유용합니다.

```
# 나이를 구간으로
df['AgeGroup'] = pd.cut(df['Age'],
                        bins=[0, 12, 18, 35, 60, 100],
                        labels=['어린이', '청소년', '청년', '중년', '노년'])

# 소득을 4분위로 (동일 개수 구간)
df['IncomeQuartile'] = pd.qcut(df['Income'], q=4,
                               labels=['Q1', 'Q2', 'Q3', 'Q4'])
```

특성 엔지니어링을 언제 하나

워크플로에서의 위치는 다음과 같습니다.

- **1차 컬럼 정리 단계**: 원본 못 쓰는 컬럼(Name, 날짜 등)에서 새 특성 뽑기. 그 다음 원본은 지우기.
- **관계 분석 후**: "이 두 컬럼을 합치면 의미 있는 새 특성이 될까?" 고민 후 조합 특성 만들기.
- **모델 학습 후**: 특성 중요도를 보고 "새 특성이 필요한가?" 판단.

주의점

- **새 특성이 많다고 좋은 게 아닙니다.** 실제로 모델 성능이 올라가는지 검증해야 합니다.
- **도메인 지식이 없으면 헛수고입니다.** "이 두 값을 곱하면 뭘 의미하는지" 알아야 특성을 만들 수 있습니다.
- **새 특성도 결측·이상치·분포 확인이 필요합니다.** 특성을 만들었다고 끝이 아닙니다.
- **train과 test에서 같은 방식으로 만들어야 합니다.** 파이프라인으로 묶으면 안전합니다.

2.9 결측값 처리

결측률에 따른 판단

결측률	처리 방법
5% 미만	행 제거 (dropna) - 데이터 손실 적음
5% ~ 30%	대체 (imputation)
30% ~ 50%	신중히 대체 + "결측 여부" 플래그 추가 고려
50% 이상	컬럼 제거 또는 결측 자체를 특성으로

수치형 대체 방법

2.3의 분포 확인이 여기서 근거가 됩니다.

분포 특성	대체 방법	이유
정규분포 (좌우 대칭)	평균 (mean)	대표값으로 적절
왜도 있음 / 이상치 있음	중앙값 (median)	극단값에 안 흔들림
시계열	앞뒤 값 (ffill/bfill), 선형 보간	시간적 연속성 유지
다변수 관계 활용	KNNImputer	비슷한 행의 값 참고

```
# 기본 대체
df['Age'].fillna(df['Age'].median(), inplace=True)

# 그룹별 대체 (더 정교)
df['Age'] = df.groupby(['Pclass', 'Sex'])['Age'] \
              .transform(lambda x: x.fillna(x.median()))
# → 1등석 남성 나이는 그 그룹의 중앙값으로

# KNN 대체
from sklearn.impute import KNNImputer
imputer = KNNImputer(n_neighbors=5)
df[num_cols] = imputer.fit_transform(df[num_cols])

# 시계열 보간
df['sensor_value'] = df['sensor_value'].interpolate(method='linear')
```

범주형 대체 방법

상황	대체 방법
최빈값이 뚜렷함	최빈값 (mode)
결측 자체가 의미	'Unknown' 새 범주로
다른 컬럼으로 추정 가능	그룹별 최빈값

```
# 최빈값
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)

# Unknown으로 (트리 모델은 이걸 별도 분기로 학습)
df['Embarked'].fillna('Unknown', inplace=True)
```

데이터 누수 함정

뭔말이냐? 데이터는 학습용/검증용으로 나뉘는데 학습용에 인코딩이 되어야 한다!
그렇기에 학습용/검증용으로 나눈 후 인코딩이 되어야한다.

> **반드시 train에서 계산한 값을 test에 적용해야 합니다.** train과 test를 각자 처리하면 test 정보가 새어들어가는 **데이터 누수**가 됩니다.

```
# 잘못된 예
X_train['Age'].fillna(X_train['Age'].median(), inplace=True)
X_test['Age'].fillna(X_test['Age'].median(), inplace=True)
# → test의 중앙값이 미래 정보에 해당

# 올바른 예
median_age = X_train['Age'].median()   # train에서만 계산
X_train['Age'].fillna(median_age, inplace=True)
X_test['Age'].fillna(median_age, inplace=True)

# 또는 SimpleImputer
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy='median')
X_train_imp = imp.fit_transform(X_train)
X_test_imp = imp.transform(X_test)
```

2.10 범주형 인코딩 인코딩은 대체가 아니당!

모델은 문자열을 직접 받지 못하므로 숫자로 바꿔야 합니다. 판단 기준은 세 가지입니다.

1. 순서가 있는가?
2. 카테고리가 몇 개인가?
3. 어떤 모델을 쓸 것인가?

판단 흐름

```
범주형 컬럼이다
├─ 순서 있음 (S/M/L, A/B/C/D/F) → Ordinal Encoding
└─ 순서 없음 (색상, 지역)
   ├─ 카테고리 ≤ 10개 → One-Hot Encoding (기본 선택)
   ├─ 10~50개 & 트리 모델 → Label Encoding OK
   └─ 50개 이상 (고카디널리티) → Target / Frequency Encoding
```

One-Hot Encoding

가장 흔한 방법입니다. 각 카테고리를 별도의 0/1 컬럼으로 만듭니다.

라이브러리
!pip install scikit-learn

```
df = pd.get_dummies(df, columns=['Sex', 'Embarked'], drop_first=True)
# drop_first=True: 다중공선성 방지 (n개 범주 → n-1개 컬럼)

# sklearn 방식 - test에 새 범주 나와도 안전
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(drop='first', sparse_output=False,
                    handle_unknown='ignore')
X_train_enc = ohe.fit_transform(X_train[['Sex']])
X_test_enc = ohe.transform(X_test[['Sex']])
```

주의점: 카테고리가 100개면 컬럼이 100개 늘어납니다(차원 폭발).

Ordinal Encoding

순서가 있는 범주에만 씁니다.

```
from sklearn.preprocessing import OrdinalEncoder

order = [['초급', '중급', '고급']]
oe = OrdinalEncoder(categories=order)
df['level_encoded'] = oe.fit_transform(df[['level']])
# 초급=0, 중급=1, 고급=2 - 거리 관계가 의미 있음
```

Label Encoder는 순서 없는 데이터에 쓰면 안 됩니다. "빨강=0, 초록=1, 파랑=2"로 하면 모델이 "파랑이 빨강의 2배"라고 오해합니다. 단, 트리 모델은 크기 비교를 하지 않아 라벨 인코딩을 써도 큰 문제는 없습니다.

Target Encoding

카테고리가 매우 많을 때(우편번호 5000개, 상품코드 만 개) 씁니다.

```
from category_encoders import TargetEncoder

te = TargetEncoder()
X_train_enc = te.fit_transform(X_train['zip_code'], y_train)
X_test_enc = te.transform(X_test['zip_code'])
# 각 우편번호를 "그 지역 평균 타깃값"으로 대체
```

과적합 위험이 크므로 K-Fold 안에서 계산해야 안전합니다.

Frequency Encoding

간단하면서 트리 모델과 궁합이 좋습니다.

```
freq = X_train['zip_code'].value_counts(normalize=True)
X_train['zip_freq'] = X_train['zip_code'].map(freq)
X_test['zip_freq'] = X_test['zip_code'].map(freq).fillna(0)
```

요약표

상황	추천 방법
이진 (M/F)	One-Hot (drop_first=True)
순서 있음 (S/M/L)	Ordinal
순서 없음, 10개 이하	One-Hot
순서 없음, 10~50개, 트리 모델	Label OK 또는 One-Hot
순서 없음, 50개 이상	Target 또는 Frequency
순서 없음, 100개 이상, 선형 모델	Target Encoding

2.11 스케일링(정규화)

1단계: 스케일링이 필요한가?

모델	스케일링
KNN, K-Means, SVM, PCA	필수 (거리 기반)
LogisticRegression, LinearRegression	필수 (계수 안정성)
신경망 (Neural Network)	필수 (학습 안정성)
RandomForest, XGBoost, LightGBM	불필요 (트리는 크기 비교만 함)

2단계: 어떤 방법을 쓸까

방법	결과	언제
StandardScaler	평균 0, 표준편차 1	기본 선택. 정규분포에 가까움
MinMaxScaler	0 ~ 1	신경망, 이미지. 이상치 없음
RobustScaler	중앙값 0, IQR 기반	이상치 많을 때
Normalizer	각 행 크기 1	텍스트 벡터
PowerTransformer	정규분포로 변환	심하게 치우친 분포

```
from sklearn.preprocessing import (StandardScaler, MinMaxScaler,
                                   RobustScaler, PowerTransformer)

# 분포 확인 후 결정
sns.histplot(df['Fare'], kde=True)
plt.show()

# 왜도 심한 경우: 로그 변환 후 StandardScaler
df['Fare_log'] = np.log1p(df['Fare'])

# 또는 PowerTransformer로 정규분포화
pt = PowerTransformer(method='yeo-johnson')
df['Fare_normal'] = pt.fit_transform(df[['Fare']])
```

3단계: 반드시 train에만 fit

```
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)    # 여기서만 fit
X_test_s = scaler.transform(X_test)           # transform만
```

2.12 지도학습과 비지도학습의 전처리 차이

전처리 흐름은 대부분 공통이지만, 몇 가지 차이가 있습니다.

단계	지도학습	비지도학습
종속변수 분리	필수 (y = df['타깃'])	없음
관계 분석	X와 y의 관계 (2.4 방법)	X끼리의 관계만
컬럼 제거 기준	4가지 모두 사용	기준 1~3만 (타깃 관계 불가)
인코딩	일반 규칙	일반 규칙
스케일링	모델별 필요 여부	**거의 항상 필요** (거리 기반)
train/test 분리	필수	보통 안 나눔

> **비지도학습에서 특히 스케일링이 중요합니다.** K-Means, PCA는 거리와 분산을 씁니다. 스케일이 다른 컬럼을 그대로 넣으면 큰 값을 가진 컬럼이 결과를 지배합니다. 소득(0~1억)과 나이(0~100)를 그대로 K-Means에 넣으면 사실상 소득으로만 군집이 나뉩니다.

2.13 시계열 전처리 — 시간 특성 만들기

시계열 데이터는 원본만으로는 모델에 못 넣습니다. "날짜"에서 여러 특성을 뽑아내는 작업이 필수입니다.

원본 데이터

```
날짜          | 가스공급량
2024-01-01   | 1250
2024-01-02   | 1180
2024-01-03   | 1320
```

이대로는 예측에 쓸 특성이 없습니다. 다음과 같이 특성을 만들어야 합니다.

A. 달력 특성 — 계절성, 요일 효과

```
df['date'] = pd.to_datetime(df['date'])
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['day'] = df['date'].dt.day
df['dayofweek'] = df['date'].dt.dayofweek       # 요일
df['is_weekend'] = (df['dayofweek'] >= 5).astype(int)
df['quarter'] = df['date'].dt.quarter
df['is_winter'] = df['month'].isin([12, 1, 2]).astype(int)
```

B. 시차(lag) 특성 — 과거값의 영향

```
df['lag_1'] = df['공급량'].shift(1)      # 하루 전
df['lag_7'] = df['공급량'].shift(7)      # 일주일 전
df['lag_365'] = df['공급량'].shift(365)  # 작년 같은 날
```

C. 롤링 통계 — 최근 추세

```
df['rolling_mean_7'] = df['공급량'].shift(1).rolling(7).mean()
df['rolling_mean_30'] = df['공급량'].shift(1).rolling(30).mean()
df['rolling_std_7'] = df['공급량'].shift(1).rolling(7).std()
```

> **반드시 shift(1) 후에 rolling을 계산합니다.** 그렇지 않으면 오늘 값이 오늘 예측에 포함되는 데이터 누수가 됩니다.

D. 외생 변수 (Exogenous)

가스공급량 예측에서 기온이 가장 결정적인 변수입니다. 외부 데이터를 결합합니다.

```
df['temperature'] = weather_df['temp']
df['heating_degree_days'] = np.maximum(18 - df['temperature'], 0)
df['is_holiday'] = df['date'].isin(holidays).astype(int)

df = df.dropna()   # lag 만들면 앞부분에 NaN 생김
```

시계열 데이터의 결측 처리

시계열은 앞뒤 값이 중요하므로 일반 대체가 아닌 **보간**을 씁니다.

```
# 앞 값으로 채우기
df['sensor'] = df['sensor'].ffill()

# 뒤 값으로 채우기
df['sensor'] = df['sensor'].bfill()

# 선형 보간
df['sensor'] = df['sensor'].interpolate(method='linear')

# 시간 기반 보간 (인덱스가 시간이어야 함)
df['sensor'] = df['sensor'].interpolate(method='time')
```

PART 3

학습 상세

전처리가 끝나면 여기부터는 상대적으로 정형화되어 있습니다. 카테고리별 대표 알고리즘을 익히고, 결과를 제대로 평가하는 법을 배웁니다.

3부. 학습 상세

3.1 train/test 분리

모델을 학습시키려면 데이터를 두 조각으로 나눠야 합니다. **train**은 학습용, **test**는 학습에 안 쓰고 나중에 성능을 검증하는 용도입니다. 보통 8:2 비율을 씁니다.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y      # 분류일 때만
)
```

stratify 옵션은 분류 문제에서 필수입니다. 클래스 비율을 유지해 나눠줍니다. 이탈 8퍼센트인 데이터에서 stratify 없이 랜덤 분리하면 test에 이탈이 15퍼센트로 몰릴 수 있습니다. 그러면 평가가 왜곡됩니다.

시계열은 다르게 나눕니다

```
# 시계열은 시간 순으로 자름
train_size = int(len(df) * 0.8)
train = df.iloc[:train_size]
test = df.iloc[train_size:]

# 교차검증도 TimeSeriesSplit
from sklearn.model_selection import TimeSeriesSplit
tscv = TimeSeriesSplit(n_splits=5)
```

3.2 지도학습 — 분류

분류 모델은 baseline부터 시작해서 성능을 올려가는 순서로 접근합니다.

Step 1: LogisticRegression (baseline)

가장 기본적인 분류 모델입니다. 이름은 "회귀"지만 실제로는 분류에 씁니다. 학습이 빠르고 해석이 쉽습니다.

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000)
model.fit(X_train_s, y_train)

y_pred = model.predict(X_test_s)
print(classification_report(y_test, y_pred))
```

Step 2: RandomForestClassifier

결정트리 여러 개를 앙상블한 모델입니다. 정형 데이터에서 baseline을 크게 뛰어넘는 성능을 보이는 경우가 많습니다. 스케일링이 필요 없다는 장점도 있습니다.

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=200,      # 트리 개수
    max_depth=None,        # 트리 깊이 (None = 무제한)
    random_state=42,
    n_jobs=-1              # 모든 CPU 사용
)
rf.fit(X_train, y_train)   # 스케일링 없이 원본 사용

y_pred = rf.predict(X_test)
print(classification_report(y_test, y_pred))

# 특성 중요도
import pandas as pd
importance = pd.Series(rf.feature_importances_, index=X_train.columns)
importance.sort_values().plot(kind='barh')
plt.show()
```

Step 3: XGBoostClassifier

정형 데이터의 실무 성능이 대부분 이걸로 결판납니다. 설치가 필요합니다(`pip install xgboost`).

```
from xgboost import XGBClassifier

xgb = XGBClassifier(
    n_estimators=300,
    learning_rate=0.1,
    max_depth=6,
    random_state=42,
    eval_metric='logloss'
)
xgb.fit(X_train, y_train)

y_pred = xgb.predict(X_test)
y_proba = xgb.predict_proba(X_test)[:, 1]   # 확률값
print(classification_report(y_test, y_pred))
```

불균형 데이터 대응

클래스 비율이 크게 다를 때(예: 이탈 3퍼센트)는 특별한 처리가 필요합니다.

```
# 방법 1: 클래스 가중치 조정
rf = RandomForestClassifier(class_weight='balanced', random_state=42)

# 방법 2: SMOTE로 소수 클래스 증강
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)
# → train만 증강. test는 원본 유지

# 방법 3: 임계값 조정
y_proba = model.predict_proba(X_test)[:, 1]
y_pred_custom = (y_proba > 0.3).astype(int)   # 기본 0.5 → 0.3으로
```

3.3 지도학습 — 회귀

Step 1: LinearRegression (baseline)

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

lr = LinearRegression()
lr.fit(X_train_s, y_train)
y_pred = lr.predict(X_test_s)

print(f"MAE:  {mean_absolute_error(y_test, y_pred):.2f}")
print(f"RMSE: {np.sqrt(mean_squared_error(y_test, y_pred)):.2f}")
print(f"R²:   {r2_score(y_test, y_pred):.3f}")
```

Step 2: RandomForestRegressor

```
from sklearn.ensemble import RandomForestRegressor

rf = RandomForestRegressor(n_estimators=200, random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)
```

Step 3: XGBoostRegressor

```
from xgboost import XGBRegressor

xgb = XGBRegressor(
    n_estimators=300, learning_rate=0.1, max_depth=6,
    random_state=42
)
xgb.fit(X_train, y_train)
y_pred = xgb.predict(X_test)
```

실제 vs 예측 시각화

회귀는 반드시 그림으로 진단합니다. 이상적으로는 점들이 대각선 위에 있어야 합니다.

```
plt.figure(figsize=(6, 6))
plt.scatter(y_test, y_pred, alpha=0.3)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()], 'r--')
plt.xlabel('실제값')
plt.ylabel('예측값')
plt.title('실제 vs 예측')
plt.show()
```

종속변수(y)의 로그 변환

회귀 문제에서 y가 심하게 오른쪽으로 치우친 경우가 흔합니다. 매출액, 집값, 소득, 요금 같은 것들이 대표적입니다. 이런 y로 그대로 학습하면 성능이 안 나오는 경우가 많습니다. 로그 변환 후 학습하고, 예측 결과를 다시 역변환하면 개선됩니다.

```
# 1. y 분포 확인
sns.histplot(y_train, kde=True)
plt.title('y 분포')
plt.show()
print(f"y 왜도: {y_train.skew():.3f}")

# 2. 왜도가 크면 (보통 |왜도| > 1) 로그 변환
y_train_log = np.log1p(y_train)   # log(1+x) - 0값도 안전
print(f"로그 변환 후 왜도: {y_train_log.skew():.3f}")

# 3. 로그 변환한 y로 학습
model.fit(X_train, y_train_log)

# 4. 예측 후 반드시 역변환
pred_log = model.predict(X_test)
pred = np.expm1(pred_log)   # 원래 스케일로 되돌림

# 5. 평가는 원래 스케일에서
print(f"MAE:  {mean_absolute_error(y_test, pred):.2f}")
print(f"RMSE: {np.sqrt(mean_squared_error(y_test, pred)):.2f}")
```

> **트리 모델은 y 로그 변환의 효과가 상대적으로 작습니다.** RandomForest, XGBoost는 분포에 덜 민감하기 때문입니다. 하지만 선형 회귀에서는 성능 개선이 큰 경우가 많으므로 y 분포는 반드시 확인해야 합니다.

3.4 비지도학습 — 군집화

K-Means 실습 (고객 세그멘테이션)

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# 1. 스케일링 (거리 기반이라 필수)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 2. K를 여러 개 시도 (엘보우 방법)
inertias = []
K_range = range(1, 11)
for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertias.append(km.inertia_)

plt.plot(K_range, inertias, marker='o')
plt.xlabel('K')
plt.ylabel('Inertia')
plt.title('엘보우 방법')
plt.show()
```

실루엣 점수로 K 재확인

```
from sklearn.metrics import silhouette_score

for k in range(2, 8):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    print(f"K={k}: 실루엣 = {score:.3f}")
# 값이 클수록 군집이 잘 나뉨
```

최종 학습과 해석

```
km = KMeans(n_clusters=5, random_state=42, n_init=10)
df['cluster'] = km.fit_predict(X_scaled)

# 군집별 특성 이해 - 이게 진짜 중요한 부분
print(df.groupby('cluster').agg({
    '연소득': 'mean',
    '소비점수': 'mean',
    '나이': 'mean',
    '방문빈도': 'mean'
}).round(1))
# → "군집 0은 고소득·저소비 = 잠재 고객"
#   "군집 3은 저소득·고소비 = 리스크 고객"
```

3.5 비지도학습 — 차원축소

PCA 기본 사용법

```
from sklearn.decomposition import PCA

# 스케일링 먼저 (필수)
X_scaled = StandardScaler().fit_transform(X)

# 2차원으로 축소 (시각화 목적)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# 얼마나 정보를 보존했는지
print(f"설명 분산 비율: {pca.explained_variance_ratio_}")
print(f"누적: {pca.explained_variance_ratio_.sum():.3f}")

# 시각화
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=df['cluster'], cmap='tab10')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.title('PCA로 본 군집')
plt.show()
```

몇 개 성분이 적당한가

```
pca_full = PCA().fit(X_scaled)
cumsum = np.cumsum(pca_full.explained_variance_ratio_)

plt.plot(cumsum, marker='o')
plt.axhline(0.9, color='r', linestyle='--')   # 90% 기준
plt.xlabel('성분 개수')
plt.ylabel('누적 설명 분산')
plt.show()
# 90% 설명하는 최소 성분 수를 채택
```

3.6 비지도학습 — 이상탐지

IsolationForest 실습

```
from sklearn.ensemble import IsolationForest

X_scaled = StandardScaler().fit_transform(X)

iso = IsolationForest(
    contamination=0.05,   # 이상치 비율 예상 (5%)
    random_state=42
)
iso.fit(X_scaled)

# -1: 이상, 1: 정상
df['anomaly'] = iso.predict(X_scaled)
df['anomaly_score'] = iso.score_samples(X_scaled)

print(df['anomaly'].value_counts())

# 가장 의심스러운 상위 10건
print(df.sort_values('anomaly_score').head(10))
```

3.7 시계열 접근

접근 A: 일반 회귀 모델로 (요즘 실무에서 자주 씀)

```
from xgboost import XGBRegressor

# 2.11에서 만든 시간 특성 사용
features = ['month', 'dayofweek', 'is_weekend', 'is_winter',
            'lag_1', 'lag_7', 'lag_365',
            'rolling_mean_7', 'rolling_mean_30',
            'temperature', 'is_holiday']

X_train = train[features]
y_train = train['공급량']
X_test = test[features]
y_test = test['공급량']

model = XGBRegressor(n_estimators=500, learning_rate=0.05, max_depth=6)
model.fit(X_train, y_train)
pred = model.predict(X_test)
```

접근 B: Prophet (설치와 사용이 쉬움)

```
from prophet import Prophet

# 컬럼명 규칙: ds(날짜), y(값)
prophet_df = train.rename(columns={'date': 'ds', '공급량': 'y'})

m = Prophet(yearly_seasonality=True, weekly_seasonality=True)
m.add_regressor('temperature')   # 외생 변수 추가 가능
m.fit(prophet_df)

future = m.make_future_dataframe(periods=len(test))
future['temperature'] = pd.concat([train['temperature'],
                                   test['temperature']]).values
forecast = m.predict(future)
```

예측 결과 시각화

```
plt.figure(figsize=(14, 5))
plt.plot(train['date'], train['공급량'], label='학습')
plt.plot(test['date'], test['공급량'], label='실제')
plt.plot(test['date'], pred, label='예측', linestyle='--')
plt.legend()
plt.show()
```

3.8 평가 지표 총정리

분류 지표

정확도(accuracy)만 보면 안 됩니다. 특히 불균형 데이터에서는 완전히 오해할 수 있습니다.

지표	의미	언제 중요
Accuracy	전체 중 맞힌 비율	클래스 균형 잡혔을 때만
Precision	양성 예측 중 실제 양성	잘못된 알람을 줄이고 싶을 때
Recall	실제 양성 중 맞힌 비율	놓치면 안 될 때 (질병 진단)
F1	Precision과 Recall의 조화평균	불균형 데이터의 기본 지표
ROC-AUC	확률 예측의 품질	임계값 무관 평가

```
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score,
                             confusion_matrix, classification_report,
                             roc_auc_score)

y_pred = model.predict(X_test)
y_proba = model.predict_proba(X_test)[:, 1]

print(f"Accuracy:  {accuracy_score(y_test, y_pred):.3f}")
print(f"Precision: {precision_score(y_test, y_pred):.3f}")
print(f"Recall:    {recall_score(y_test, y_pred):.3f}")
print(f"F1:        {f1_score(y_test, y_pred):.3f}")
print(f"ROC-AUC:   {roc_auc_score(y_test, y_proba):.3f}")
print(classification_report(y_test, y_pred))

# 혼동행렬
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('예측')
plt.ylabel('실제')
plt.show()
```

회귀 지표

지표	의미	특징
MAE	평균 절대 오차	해석 쉬움. 같은 단위
RMSE	제곱근 평균 제곱 오차	큰 오차에 페널티. 이상치 민감
R^2	결정계수 (0~1)	"몇 %를 설명하나"
MAPE	평균 절대 백분율 오차	"평균 몇 % 틀림". 시계열

```
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)
mape = np.mean(np.abs((y_test - y_pred) / y_test)) * 100

print(f"MAE:  {mae:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R²:   {r2:.3f}")
print(f"MAPE: {mape:.2f}%")
```

비지도학습 평가

정답이 없어 "정확도" 개념이 없습니다.

- **군집화**: 실루엣 점수(Silhouette Score), Calinski-Harabasz 지수. 그리고 도메인 지식으로 해석.
- **차원축소**: 설명 분산 비율(explained_variance_ratio_). 시각화 품질.
- **이상탐지**: 정답 라벨이 일부 있으면 지도학습 지표 사용. 없으면 도메인 검증.

> **비지도학습 결과는 "의사결정 재료"이지 답이 아닙니다.** 학생들이 자주 "K-Means 결과가 정답인가요?"라고 묻는데, 정답은 없습니다. 결과를 도메인 관점에서 해석하고 활용할 수 있느냐가 중요합니다.

3.9 하이퍼파라미터 튜닝

모델의 성능을 마지막으로 짜내는 단계입니다. GridSearchCV가 가장 흔한 방법입니다.

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [100, 200, 400],
    'max_depth': [None, 5, 10, 20],
    'min_samples_split': [2, 5, 10]
}

grid = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,              # 5-fold 교차검증
    scoring='f1',      # 평가 지표
    n_jobs=-1,
    verbose=1
)
grid.fit(X_train, y_train)

print(f"최적 파라미터: {grid.best_params_}")
print(f"최적 F1:       {grid.best_score_:.3f}")

# 최적 모델로 test 평가
best_model = grid.best_estimator_
y_pred = best_model.predict(X_test)
print(classification_report(y_test, y_pred))
```

튜닝 순서 팁

- 많은 조합 한 번에 넣지 마세요. 시간이 폭발합니다. 3^3 = 27개 정도로 시작.
- 중요한 파라미터부터: 트리 계열은 `n_estimators`, `max_depth`가 우선.
- 큰 범위로 대략 잡고 → 좁은 범위로 정밀하게, 두 단계로 진행.
- 더 빠른 대안: `RandomizedSearchCV` (조합을 랜덤 샘플링).

3.10 파이프라인으로 정리하기

전처리와 학습을 한 객체로 묶으면 코드가 깔끔해지고 데이터 누수 위험도 줄어듭니다.

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier

num_cols = ['Age', 'Fare']
cat_cols = ['Sex', 'Embarked']

preprocessor = ColumnTransformer([
    ('num', StandardScaler(), num_cols),
    ('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols)
])

pipe = Pipeline([
    ('prep', preprocessor),
    ('model', RandomForestClassifier(random_state=42))
])

pipe.fit(X_train, y_train)
print(f"Test score: {pipe.score(X_test, y_test):.3f}")

# GridSearchCV에도 그대로 넣을 수 있음
param_grid = {
    'model__n_estimators': [100, 200],
    'model__max_depth': [None, 10]
}
grid = GridSearchCV(pipe, param_grid, cv=5)
grid.fit(X_train, y_train)
```

3.11 자주 겪는 함정

함정 1: 데이터 누수

test 정보가 train에 스며드는 것입니다. 성능이 이상하게 좋게 나오면 의심해야 합니다.

```
# 잘못된 예 - 전체에 스케일링 후 분리
scaler = StandardScaler()
X_scaled_all = scaler.fit_transform(X)
X_tr, X_te, y_tr, y_te = train_test_split(X_scaled_all, y)

# 올바른 예 - 분리 후 train에만 fit
X_tr, X_te, y_tr, y_te = train_test_split(X, y)
scaler = StandardScaler()
X_tr = scaler.fit_transform(X_tr)
X_te = scaler.transform(X_te)
```

함정 2: 불균형 데이터의 accuracy 함정

```
from sklearn.dummy import DummyClassifier

# 아무 예측이나 하는 모델
dummy = DummyClassifier(strategy='most_frequent')
dummy.fit(X_tr, y_tr)
print(f"어이없는 accuracy: {dummy.score(X_te, y_te):.3f}")
# → 이탈 3%인 데이터에서 accuracy 0.97 나옴
#   근데 이탈자는 한 명도 못 맞힘

print(classification_report(y_te, dummy.predict(X_te)))
# → recall = 0, F1 = 0
```

함정 3: 과적합 진단

```
from sklearn.model_selection import learning_curve

train_sizes, train_scores, val_scores = learning_curve(
    RandomForestClassifier(random_state=42),
    X_tr, y_tr, cv=5,
    train_sizes=np.linspace(0.1, 1.0, 10)
)

plt.plot(train_sizes, train_scores.mean(axis=1), 'o-', label='train')
plt.plot(train_sizes, val_scores.mean(axis=1), 'o-', label='validation')
plt.xlabel('학습 데이터 크기')
plt.ylabel('정확도')
plt.legend()
plt.title('학습 곡선 - 과적합 진단')
plt.show()
# train 1.0, val 0.75면 과적합 → max_depth 낮추기
```

함정 4: 시계열의 랜덤 분리

미래로 과거를 예측하는 꼴이 됩니다. 반드시 시간 순으로 자릅니다.

함정 5: 시계열 rolling에서 shift 빠뜨림

오늘 값이 오늘 예측에 포함되는 누수. 반드시 `.shift(1).rolling(...)` 순서로.

함정 6: 회귀에서 잔차 진단 생략

회귀 모델을 학습시켰다면 예측값만 보지 말고 **잔차(실제값 - 예측값)**도 확인해야 합니다. 잔차의 패턴이 모델의 문제를 알려줍니다.

```
model.fit(X_train, y_train)
pred = model.predict(X_train)
residuals = y_train - pred

# 진단 1: 잔차의 분포 - 정규분포에 가까워야 함
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.histplot(residuals, kde=True, ax=axes[0])
axes[0].set_title('잔차 분포')
axes[0].axvline(0, color='red', linestyle='--')

# 진단 2: 잔차 Q-Q Plot
from scipy import stats
stats.probplot(residuals, dist='norm', plot=axes[1])
axes[1].set_title('잔차 Q-Q Plot')
plt.tight_layout()
plt.show()

# 진단 3: 잔차 vs 예측값 - 등분산성 확인
plt.figure(figsize=(8, 5))
plt.scatter(pred, residuals, alpha=0.3)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel('예측값')
plt.ylabel('잔차')
plt.title('잔차 vs 예측값')
plt.show()
```

진단 결과 해석:

- **잔차 분포가 0 중심의 정규분포에 가까움** → 모델이 잘 학습됨
- **잔차가 한쪽으로 치우침** → 모델이 체계적으로 과대·과소 예측 중. 특성 추가 필요
- **잔차 vs 예측값에 패턴이 보임 (깔때기 모양 등)** → 등분산성 위반. 로그 변환 또는 비선형 모델 필요
- **잔차 vs 예측값이 무작위로 흩어짐** → 좋은 모델

선형 회귀의 통계적 가정(정규성, 등분산성, 독립성)을 확인하는 것이 잔차 진단의 원래 목적입니다. 실무 ML에서는 자주 생략하지만, 통계적 해석이 필요하거나 모델 개선 실마리를 찾을 때 매우 유용합니다.

P A R T 4

실전 워크플로와 AI 활용법

자기 데이터를 앞에 두고 진행하는 실전 흐름. 그리고 AI를 답 주는 기계가 아닌 조사원으로 부리는 방법.

4부. 실전 워크플로와 AI 활용법

4.1 실전 워크플로 11단계

이 워크플로는 어떤 프로젝트든 큰 틀에서 동일합니다. 프린트해서 옆에 두고, 자기 데이터를 만날 때마다 순서대로 진행하세요. 2부의 실용 순서(2.3)를 그대로 반영한 흐름입니다.

> **11단계 진행표**
>
> 1. **데이터 훑어보기.** shape, head, info, describe, isnull.sum. 데이터의 인상을 잡습니다.
> 2. **문제 판단.** 지도/비지도/강화 결정. 시계열 여부 확인. 판단 시트를 채웁니다.
> 3. **종속변수와 독립변수 분리.** y와 X를 명확히 나눕니다. (비지도는 이 단계 건너뜀)
> 4. **1차 컬럼 정리 + 특성 엔지니어링.** 명백히 지울 것(식별자, 결측 다수, 상수) 정리. 원본 못 쓰지만 가공하면 유용한 컬럼에서 새 특성 뽑기 (Name → Title, 날짜 → 연/월/요일 등).
> 5. **분포 확인.** 남은 수치형 컬럼마다 히스토그램, Q-Q 플롯, 왜도.
> 6. **관계 분석.** X와 y의 관계 (수치×수치는 산점도, 수치×범주는 boxplot 등). 그리고 X끼리의 관계 (상관관계 히트맵, VIF).
> 7. **2차 컬럼 정리.** 타깃과 관계 약한 것 제거, 다중공선성 대응.
> 8. **결측·인코딩·스케일링.** 각 컬럼별 판단대로 처리. 반드시 train에만 fit.
> 9. **train/test 분리.** 분류면 stratify, 시계열이면 시간 순.
> 10. **모델 학습.** baseline → 트리 → XGBoost 순서.
> 11. **평가와 해석.** 문제 성격에 맞는 지표로. 단순 숫자만 보지 말고 시각화까지.

시간 배분 예시

실무에서는 이 정도 비율이 나옵니다. 처음 배우는 사람은 여기서 오해가 생깁니다. "모델링에 시간을 많이 쓸 줄 알았는데" 하는 반응이 흔합니다.

단계	시간 비율
1~3단계 (탐색·판단·분리)	15%
4단계 (1차 정리 + 특성 엔지니어링)	15%
5~7단계 (분포·관계·2차 정리)	20%
8단계 (전처리 마무리)	20%
9~10단계 (분리·학습)	10%
11단계 (평가·해석)	20%

4.2 AI에게 물을 때 반드시 줘야 하는 정보

여러분이 실무에 나가면 매번 새 데이터를 만납니다. 모든 케이스를 미리 배울 수 없기 때문에, 결국 생성형 AI에게 물으며 문제를 풀게 됩니다. 이 능력을 훈련하는 게 이 가이드의 가장 실용적인 부분입니다.

AI는 여러분의 데이터를 못 봅니다. 다음 네 가지 정보를 안 주면 뜬구름 잡는 답이 나옵니다.

필수 4종 세트

1. **목표.** 뭘 예측·분석하고 싶은지 한 문장으로.
2. **데이터 모양.** 몇 행 × 몇 열, 컬럼 이름과 타입, 예시 3~5행.
3. **타깃.** 예측할 컬럼과 그 값의 형태(숫자? 카테고리? 몇 개?).
4. **제약.** 사용할 도구(sklearn/pandas), 시간, 성능 요구사항.

추가로 유용한 것

- `df.info()` 결과 붙이기
- `df.describe()` 결과 붙이기
- `df.isnull().sum()` 결과 붙이기
- 데이터의 도메인 배경 (어떤 업계? 어떻게 수집?)

이 정도만 붙여도 답 품질이 눈에 띄게 좋아집니다.

4.3 단계별 프롬프트 템플릿

수업 실습 때 이 템플릿들을 프린트해서 나눠 씁니다. 매 단계마다 채워서 AI에게 물어보세요.

템플릿 1 — 문제 판단

```
[상황]
저는 ○○업계의 △△ 데이터로 □□를 하고 싶습니다.

[데이터]
- 행 수: 2000행
- 컬럼: 날짜, 지역, 기온, 요일, 공휴일여부, 가스공급량
- 예시 3행:
  2024-01-01, 서울, -5, 월, N, 1250
  2024-01-02, 서울, -3, 화, N, 1180
  2024-01-03, 서울,  0, 수, N, 1320

[예측 대상]
가스공급량 (숫자, 연속값)

[질문]
이 문제는 지도학습/비지도학습/강화학습 중 어디에 해당하나요?
시계열로 다뤄야 하나요, 일반 회귀로 접근해도 되나요?
판단 근거도 알려주세요.
```

템플릿 2 — 전처리 판단

```
[데이터 상태]
df.info() 결과:
(붙여넣기)

df.describe() 결과:
(붙여넣기)

df.isnull().sum() 결과:
(붙여넣기)

[타깃] 이탈여부 (0/1)

[질문]
각 컬럼에 대해 표로 정리해 주세요:
1. 제거해야 하나? (이유)
2. 유지하면 어떻게 전처리해야 하나?
3. 특성 엔지니어링으로 새로 만들 수 있는 게 있나?
```

템플릿 3 — 결측값 처리

```
[컬럼 정보]
- 컬럼명: Age
- 타입: 수치형
- 결측률: 20%
- 분포: 왼쪽 치우침 (왜도 1.3)
- 이상치: 80세 이상 소수 존재

[관련 컬럼]
Pclass(등급), Sex 컬럼이 나이와 상관이 있어 보입니다.

[질문]
1. 결측값 처리 방법들(삭제/평균/중앙값/최빈값/그룹별/KNN)의 장단점
2. 이 데이터에 가장 적합한 방법과 이유
3. 실제 pandas 코드
```

템플릿 4 — 인코딩 선택

```
[내 범주형 컬럼]
- Sex: 2개 값 (M/F)
- Region: 17개 값 (전국 시도)
- Product_Code: 3200개 값 (상품 코드)
- Grade: 5개 값 (A/B/C/D/F, 순서 있음)

[사용 모델] XGBoost와 LogisticRegression 둘 다 시도 예정

[질문]
각 컬럼에 어떤 인코딩(원핫/라벨/오디널/타깃/빈도)이 좋을까요?
컬럼별로 추천 방법과 이유, 코드를 알려주세요.
```

템플릿 5 — 스케일링 판단

```
[내 상황]
- 모델: RandomForest / XGBoost / LogisticRegression 중 선택 예정
- 수치 컬럼 분포:
  - Age: 정규분포에 가까움 (0~90)
  - Fare: 심한 오른쪽 치우침, 이상치 있음 (0~500)
  - Distance: 균일 분포 (0~100)

[질문]
1. 각 모델별로 스케일링 필요 여부
2. 필요하다면 각 컬럼에 어떤 방법(Standard/MinMax/Robust/log변환)이 좋을지
3. 이유와 코드
```

템플릿 6 — 모델 선택

```
[문제] 지도학습 이진분류. 이탈 예측.

[데이터]
- 5000행 × 25컬럼
- 클래스 비율: 이탈 8%, 유지 92% (불균형)
- 특성: 수치 15개 + 범주 10개

[제약]
- sklearn만 사용 (XGBoost는 가능)
- 학습 시간 5분 이내
- 모델 해석이 필요함 (마케팅팀 보고용)

[질문]
어떤 알고리즘 3개를 후보로 삼을지 추천해 주세요.
각각의 장단점, 불균형 데이터 대응 방법,
하이퍼파라미터 초기값도 포함해서.
```

템플릿 7 — 코드 요청

```
[요청]
아래 조건으로 sklearn 코드 작성:

- 데이터: X_train, X_test, y_train, y_test 이미 나눠져 있음
- 전처리: 수치는 StandardScaler, 범주는 OneHotEncoder
- 모델: RandomForestClassifier
- 평가: accuracy, F1, ROC-AUC, confusion_matrix
- 파이프라인으로 묶기
- 각 줄에 한글 주석

[출력 형식]
전체 코드 한 블록으로. 실행 순서대로.
```

템플릿 8 — 성능 개선

```
[현재 상황]
모델: RandomForestClassifier(기본값)
- train accuracy: 0.99
- test accuracy: 0.72
- test F1: 0.31 (이탈 클래스)

[데이터]
- 이탈 8% (불균형)
- 5000행

[질문]
1. 지금 뭐가 문제인지 진단 (과적합? 불균형? 특성 부족?)
2. 시도해볼 개선 방법 3가지, 우선순위대로
3. 각 방법의 실제 코드
```

템플릿 9 — 에러 디버깅

```
[상황] 이 코드를 실행했는데 에러가 났습니다.

[코드]
(붙여넣기)

[에러 메시지]
(전체 트레이스백 붙여넣기)

[데이터 상태]
X_train.shape: (4000, 15)
X_train.dtypes: (붙여넣기)

[질문]
1. 에러 원인
2. 해결 방법
3. 왜 이 에러가 났는지 개념 설명
```

템플릿 10 — 관계 분석

```
[내 데이터]
- 종속변수(y): 이탈여부 (0/1, 범주형)
- 독립변수(X):
  - 가입기간 (수치형)
  - 월사용액 (수치형)
  - 성별 (범주형)
  - 지역 (범주형, 17개)

[질문]
각 독립변수와 종속변수의 관계를 어떻게 확인해야 하나요?
- 어떤 그래프를 그려야 하나요?
- 어떤 통계 검정이 적절한가요?
- 각각의 코드도 포함해서 정리해 주세요.
```

4.4 AI 답변 검증 4단계

AI가 준 답을 그대로 믿으면 안 됩니다. 이건 반드시 몸에 새겨야 하는 습관입니다.

1단계: 실행해 봅니다

코드는 무조건 돌립니다. 에러가 나면 바로 알 수 있습니다. 결과 숫자가 상식적인지도 확인합니다. 정확도 100퍼센트가 나왔다면 무언가 잘못된 겁니다.

2단계: 의도한 대로 됐는지 확인합니다

```
# AI가 준 코드 실행 후
print(X_train.head())         # 진짜 전처리가 됐나?
print(X_train.dtypes)          # 타입이 맞나?
print(model.classes_)          # 클래스 순서가 맞나?
```

3단계: 이유를 다시 물어봅니다

- "왜 중앙값을 썼나요? 평균은 왜 안 되나요?"
- "왜 이 모델을 골랐나요? 다른 후보는요?"
- "이 하이퍼파라미터는 왜 이 값인가요?"

AI가 이유를 제대로 설명하지 못하면 답이 틀렸을 가능성이 높습니다.

4단계: 반대로도 물어봅니다

- "이 접근의 단점은 무엇인가요?"

- "이 방법이 잘 안 통하는 경우는 언제인가요?"
- "이 답을 반박한다면 어떤 근거를 들 수 있나요?"

빨간 신호 (경고 사인)

다음이 나오면 의심하세요.

- 코드에 존재하지 않는 함수나 파라미터 (환각)
- accuracy 99%인데 이유 설명이 얕음
- "일반적으로 좋다"는 근거 없는 단정
- 데이터 특성 무시하고 무조건 특정 방법 추천
- 데이터 누수가 있는 코드 (test에 fit 하는 등)

4.5 위험한 프롬프트 패턴

나쁜 예 1: 데이터 정보 없이 묻기

> "머신러닝 코드 짜줘"
> "내 데이터 분류해줘"

AI가 뜬구름 답변을 내놓습니다. 학생 본인도 뭘 배웠는지 모릅니다.

나쁜 예 2: 판단은 안 하고 답만 요구

> "이거 지도학습이야 비지도학습이야?"
> (데이터 설명 없이)

학생이 판단력을 기를 기회를 놓칩니다.

나쁜 예 3: 통째로 맡기기

> "타이타닉 데이터로 처음부터 끝까지 코드 짜줘"

학생은 아무것도 배우지 못합니다. 나눠서 물어야 각 단계를 이해합니다.

나쁜 예 4: 검증 없이 붙여넣기

> AI 답변 그대로 → 노트북에 붙여넣기 → 안 돌아가면 다시 물음
> → 계속 반복만 하고 실력이 안 늚

한 문장 원칙

> **판단은 여러분이 하고, AI는 조사원처럼 씁니다.** 답을 주는 존재가 아니라 정보를 정리해 주는 존재로 대하세요.

4.6 실전 세션 흐름 예시

수업 두 시간 세션의 예시입니다. 자기 프로젝트를 진행할 때도 이 흐름을 따라 하면 좋습니다.

단계	시간	내용
1	10분	문제 소개. "가스공급량 예측을 한다."
2	15분	데이터 훑어보기. df.info, head, describe 직접 실행.
3	15분	템플릿 1 채워서 AI에게 판단 요청.
4	15분	AI 답변 함께 읽기. "왜 이 답이 나왔을까" 토론.
5	20분	템플릿 2로 전처리 요청. AI 답변 검증.
6	25분	코드 실행과 결과 해석.
7	20분	회고. 서로의 프롬프트 비교. "누구의 프롬프트가 좋은 답을 뽑았는가."

이 방식의 강점은 **학생마다 데이터가 달라도 같은 흐름으로 진행할 수 있다**는 점입니다. 미니 프로젝트에도 그대로 적용할 수 있습니다.

4.7 요약 카드 — 한 장 정리

AI에게 ML 도움 요청 5원칙

1. **데이터를 보여줘라.** shape, dtypes, head(), describe() 붙여넣기
2. **목표를 한 문장으로 말해라.** "○○컬럼(형태: X)을 예측하고 싶다"
3. **판단부터 요청해라.** 코드 먼저 요구하지 말 것. "무슨 학습이야? 왜?"
4. **답을 검증해라.** 실행, 이유 재질문, 반대 관점 요청
5. **나눠서 물어라.** 판단 → 전처리 → 모델 → 평가 → 개선. 통째로 요청하면 아무것도 못 배움

전처리 판단 시트

1. 컬럼 하나씩 보면서 — 식별자? 결측 50%↑? 값 거의 하나? 타깃과 관계 있음?
2. 남은 컬럼의 결측 — 5% 미만이면 행 제거, 그 외는 왜도 확인 후 평균/중앙값
3. 범주형 인코딩 — 순서 있음? Ordinal, 10개 이하? One-Hot, 그 외 Label(트리) 또는 Target
4. 스케일링 — 트리 모델은 안 함, 그 외는 분포에 따라 Standard/Robust/log변환

학습·평가 원칙

- baseline부터 시작 (LogisticRegression / LinearRegression)
- 정형 데이터의 강자는 트리 계열 (RandomForest, XGBoost)
- 불균형 데이터에서 accuracy 믿지 말 것. F1이나 recall
- 모든 스케일러·인코더는 train에만 fit
- 시계열은 시간 순 분리. rolling 전에 shift(1)
- 비지도학습 결과는 답이 아니라 재료

4.8 마무리 — 자기 프로젝트 시작하기

머신러닝은 완벽한 정답이 있는 분야가 아닙니다. 같은 데이터를 열 명이 처리하면 열 가지 접근이 나옵니다. 중요한 건 **내가 왜 이 선택을 했는지 설명할 수 있는가**입니다.

이 가이드를 옆에 두고 자기 데이터로 여러 번 반복해 보세요. 처음엔 판단 하나하나가 어렵고 오래 걸립니다. 세 번째 프로젝트쯤 되면 판단 흐름이 자연스러워집니다. 그때부터가 진짜 시작입니다.

첫 프로젝트를 위한 데이터 추천

- **Titanic (분류 기본)** — Kaggle에서 CSV 다운로드

- **California Housing (회귀 기본)** — sklearn 내장 (`fetch_california_housing`)
- **Iris (분류 초심자)** — sklearn 내장
- **Mall Customers (군집화)** — Kaggle에서 CSV 다운로드
- **업무 데이터** — 가장 좋은 학습 자료는 자기 업무 데이터

막힐 때 돌아올 곳

- 판단이 흐릿할 때 → **1.9 판단 프레임워크**
- 전처리 순서가 헷갈릴 때 → **2.3 전처리의 실용 순서**
- 분포·정규성 확인이 헷갈릴 때 → **2.4 분포 확인과 정규성 판단**
- 어떤 컬럼 지울지 고민될 때 → **2.7 컬럼 제거 판단 4가지 기준**
- 원본 컬럼을 살릴 방법 → **2.8 특성 엔지니어링**
- 결측·인코딩·스케일링이 헷갈릴 때 → **2.9, 2.10, 2.11**
- 어떤 모델 쓸지 모를 때 → **3.2, 3.3** baseline 순서
- 성능이 안 나올 때 → **3.11 함정 진단**
- AI에게 물을 때 → **4.3 프롬프트 템플릿**

마지막 한마디

막히면 AI에게 물으세요. 단, **판단은 항상 여러분 몫**이라는 걸 잊지 마세요. AI가 코드를 짜주는 시대에도, 좋은 질문을 던지고 답을 검증할 수 있는 사람과 그렇지 않은 사람의 격차는 오히려 커집니다.

여러분이 이 가이드를 덮을 즈음이면, 자기 데이터를 앞에 놓고 처음 해야 할 일이 무엇인지 알게 될 겁니다. 그거면 충분합니다. 나머지는 하다 보면 저절로 늘어납니다.

— 끝 —